Open in app ↗

Medium Search Write 15

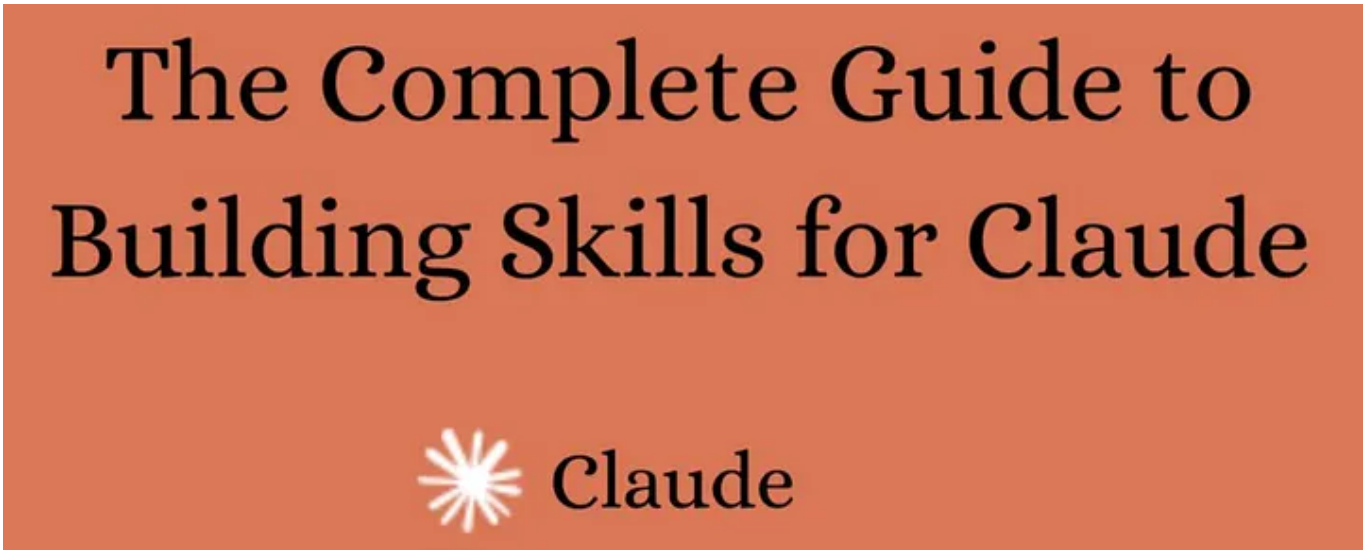
Member-only story

I Read the 33-Page Claude Skills Guide So You Don’t Have To

Civil Learning Follow 9 min read · 5 days ago

310 5

Anthropic dropped a full guide on building Skills for Claude. It’s thorough. It’s also 33 pages. I broke it down into what actually matters. By the end of this, you’ll know what Skills are, how they work, and how to build one from scratch.



What even is a Skill?

Think of it as a folder that teaches Claude how to perform a specific task.

Instead of explaining your workflow every single time you open a new chat, you package it once. Claude learns it, uses it automatically, and you stop repeating yourself.

Here's what goes in the folder:

```
┌──────────────────────────────────────┐
│     your-skill-name/                 │
│                                      │
│   ┌──────────────────┐               │
│   │   SKILL.md       │ ←── Required  │
│   │  (Instructions)  │               │
│   └──────────────────┘               │
│                                      │
│   ┌──────────────────┐               │
│   │   scripts/       │ ←── Optional  │
│   │  - Python files  │               │
│   │  - Bash scripts  │               │
│   └──────────────────┘               │
│                                      │
│   ┌──────────────────┐               │
│   │  references/     │ ←── Optional  │
│   │  - Documentation │               │
│   │  - API guides    │               │
│   └──────────────────┘               │
│                                      │
│   ┌──────────────────┐               │
│   │   assets/        │ ←── Optional  │
│   │  - Templates     │               │
│   │  - Images        │               │
│   └──────────────────┘               │
└──────────────────────────────────────┘
```

The entire system operates at three load levels. Level 1 is the YAML frontmatter at the top of SKILL.md — always loaded, tells Claude what the skill is for. Level 2 is the main body, loaded when Claude thinks it's relevant. Level 3 is the extra files, pulled in only when Claude needs the details. Keeps context small until it matters.

Skills vs MCP — what's the difference?

People confuse these two. Here's the clearest way I can explain it:

```
┌──────────────────┐          ┌──────────────────┐
│      MCP         │          │     SKILLS       │
│  "The Tools"     │          │  "The Process"   │
│                  │          │                  │
│  • Connections   │          │  • How to use    │
│  • Data access   │    +     │  • Workflows     │
│  • Tool calls    │          │  • Best practice│
│  • Real-time     │          │  • Domain know.  │
│                  │          │                  │
│  "What Claude    │          │  "How Claude     │
│   CAN do"        │          │   SHOULD do"     │
└──────────────────┘          └──────────────────┘
          │                            │
          └────────────────────────────┘
                       │
                       ▼
          ┌────────────────────┐
          │  Complete Solution  │
          │                    │
          │  Tool Access       │
          │       +            │
          │  Knowledge Layer   │
          │       =            │
          │  Reliable Results  │
          └────────────────────┘
```

MCP gives Claude the tools. Skills teach Claude how to use them.

You hire a contractor and hand them access to your workshop. That's MCP. They still need blueprints and a process. That's the skill.

Without the skill, users connect your MCP integration and immediately ask "now what?" With it, Claude already knows.

Chapter 1: Planning your skill

Don't build a skill just because you can. Build one because a specific problem keeps coming up.

Frame it like this — using customer support ticket analysis as an example:

```
┌──────────────────────────────────────────────┐
│            USE CASE DEFINITION               │
└──────────────────────────────────────────────┘
                      │
          ┌───────────┼───────────┐
          │           │           │
          ▼           ▼           ▼
     ┌─────────┐  ┌─────────┐  ┌─────────┐
     │ What's  │  │  What   │  │  What   │
     │  the    │  │ tools   │  │ domain  │
     │  goal?  │  │needed?  │  │ knowl?  │
     └─────────┘  └─────────┘  └─────────┘
          │           │           │
          └───────────┼───────────┘
                      │
                      ▼
          ┌──────────────────────┐
          │   CONCRETE STEPS     │
          │   1. Action 1        │
          │   2. Action 2        │
          │   3. Action 3        │
          └──────────────────────┘
                      │
                      ▼
          ┌──────────────────────┐
```

```
          |   EXPECTED RESULT    |
          └──────────────────────┘
```

Trigger: User says “analyze my support tickets” or “show me customer pain points”

Steps:

1. Pull last 30 days of tickets from Zendesk
2. Group by category and severity
3. Identify patterns in complaints
4. Generate summary with action items
5. Create presentation with findings

Result: Complete analysis deck ready to share with the team

Anthropic identified three types of skills people actually build:

```
                 ┌──────────────┐
                 |  SKILL TYPES    |
                 └──────┬───────┘
                        |
        ┌───────────────┼───────────────┐
        |               |               |
        ▼               ▼               ▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
|  CATEGORY 1   | |  CATEGORY 2   | |  CATEGORY 3   |
|   Making      | |   Running     | |     MCP       |
|   Stuff       | |   Processes   | |  Enhancement  |
└──────┬───────┘ └──────┬───────┘ └──────┬───────┘
       |                |                |
   ┌───▼───┐        ┌───▼───┐        ┌───▼───┐
```

```
| Creates |        | Multi-  |        | Adds    |
| Output  |        | step    |        |workflow |
|         |        |workflow |        |guidance |
|• Docs   |        |         |        |to tools |
|• Code   |        |• Order  |        |         |
|• Designs|        | matters |        |• Query  |
|• Content|        |• Gates  |        | help    |
|         |        |• Loops  |        |• Best   |
|Uses     |        |         |        | practice|
|Claude's |        |Uses     |        |         |
|built-in |        |built-in |        |Requires |
|tools    |        |+ maybe  |        |MCP      |
|         |        |MCP      |        |server   |
  ────────          ────────           ────────
```

Category 1 — Making Stuff: Creates output. Marketing copy, code, slide decks. Example: a skill that drafts investor emails. It knows your metrics, your tone, your format.

Category 2 — Running Processes: Automates multi-step workflows where order matters. Example: a weekly content planning skill that checks last week's performance, suggests topics, builds the calendar, and assigns tasks.

Category 3 — MCP Enhancement: Turns raw tool access into actual workflows. You built a database connector. Now build a skill that knows how to query it correctly.

Chapter 2: Building it

Your folder needs this exact structure:

```
email-campaign-builder/
├── SKILL.md
├── scripts/
|   └── validate_links.py
```

```
├── references/
│   └── brand-voice-guide.md
└── assets/
    └── email-templates/
```

Rules that matter: `SKILL.md` must be spelt exactly like that — capital S, capital K, capital L. Folder names use dashes (`email-campaign-builder` ✅, not spaces or underscores). Don't put a `README.md` inside the skill folder.

The SKILL.md file

```
┌──────────────────────────────────────┐
│          SKILL.md FILE               │
├──────────────────────────────────────┤
│  ---                                 │ ◀── Delimiter
│  name: your-skill-name               │ ◀── Required
│  description: What it does.          │ ◀── Required
│    Use when user asks to X.          │      (What + When)
│  ---                                 │ ◀── Delimiter
│                                      │
│  # Your Skill Name                   │
│                                      │
│  ## Instructions                     │
│                                      │
│  Step 1: Do this                     │
│  Step 2: Then this                   │
│                                      │
│  ## Examples                         │
│  ...                                 │
│                                      │
│  ## Troubleshooting                  │
│  ...                                 │
└──────────────────────────────────────┘
```

The minimum viable frontmatter:

name: email-campaign-builder
description: Creates email campaigns for product launches. Use when user wants t
```
That's it for a starting point.
### Writing descriptions that actually trigger
The description is how Claude decides whether to load your skill. Get it wrong a
**Good:**
```
description: Generates SQL queries for the customer analytics database. Use when
```
**Bad:**
```
description: Helps with database stuff.
```
Too vague. Claude has no idea when to use this.
### Writing instructions
Write like you're training someone new, not writing documentation.
Instead of: *"Ensure data integrity"*
Write: *"Run `python scripts/check_data.py` and verify these three outputs: no m
Include what happens when things break: *"If the API returns a 429 error, wait 6
Heavy reference docs go in `references/`. Keep `SKILL.md` focused on the core wo
---
## Chapter 3: Testing it
Three ways to test. Pick based on your situation.
**Manual** - open Claude.ai and try it. Fast, no setup. Good for early drafts.
**Scripted** - write test cases in Claude Code, run them as you iterate.
**API** - build full evaluation suites for production.
Personal use? Manual is fine. Deploying to 1,000 users? Go deeper.
**What Anthropic actually recommends:** Focus on one hard task first. Get Claude
Three things to check:
**Does it trigger right?** Test obvious requests, test variations, and test unre
**Does it work?** Run the full workflow, check outputs, test edge cases like emp
**Is it better than nothing?** Compare with and without the skill. Count message
Real example of what the difference looks like:
Without skill: 12 messages, 3 failed API calls, user has to explain the process.
With skill: 3 messages, 0 failed calls, Claude already knows the process.
### Using skill-creator
There's a built-in skill called `skill-creator`. It helps you build other skills
It also reviews skills you've already written - just say "review this skill and
### When things go wrong
**Skill doesn't load:** description is too generic. Add specific trigger phrases
**Skill loads for everything:** description is too broad. Add negative triggers:
**Skill loads but fails:** instructions are unclear. Add more specifics, include
---
## Chapter 4: Patterns worth stealing
These come from real skills built by early users. Use them as starting points.
### Pattern 1: Sequential workflows
Use this when steps must happen in a specific order.
```

```
Step 1: Verify transaction exists
Call: get_transaction(transaction_id)
Check: transaction.status == "completed"
Step 2: Check refund eligibility
Rule: Refund allowed if < 30 days
Rule: Product not marked as final sale
Step 3: Calculate refund amount
If partial return: calculate prorated amount
If full return: use original transaction amount
Step 4: Process refund
Call: create_refund(transaction_id, amount)
Verify: refund.status == "succeeded"
Step 5: Send confirmation
Template: refund_confirmation_email
Include: original order details, refund amount, timeline
```

Key techniques: explicit dependencies between steps, validation before proceedin

Pattern 2: Multi-tool coordination

Use this when you need multiple services working together.

```
Phase 1: Content Creation (Internal)
- Draft blog post based on outline
- Generate 3 social media versions
- Create meta descriptions and tags
Phase 2: Asset Generation (MCP: Midjourney)
- Generate header image based on topic
- Create 3 social media graphics
- Save to asset library
Phase 3: Publication (MCP: WordPress)
- Upload blog post with formatting
- Attach header image
- Set publication date and categories
Phase 4: Social Scheduling (MCP: Buffer)
- Queue social posts
- Attach generated graphics
- Stagger timing across platforms
Phase 5: Team Notification (MCP: Slack)
- Post to #marketing channel
- Include preview link
- Tag relevant team members
```

Pattern 3: Iterative improvement

Use this when output gets better through refinement.

```
Initial Draft:
1. Analyze data structure
2. Choose appropriate chart type
3. Generate first version
Quality Check:
```

```
1. Run validation script
2. Check for: missing labels, unclear axes, color accessibility issues
3. List improvements needed
Refinement:
1. Address each issue
2. Regenerate affected sections
3. Re-validate
Repeat until:
- All labels clear
- Data accurately represented
- Visual hierarchy works
- Accessible color scheme used
Finalize:
- Export in multiple formats
- Generate embed code
- Save to library
```

Chapter 5: YAML and troubleshooting

The YAML frontmatter trips people up more than anything else.

```
# Wrong - missing dashes
name: my-skill
description: Does things

# Wrong - quotes not closed
description: "Does things
# Right
---
name: my-skill
description: Does things
---
```

Skill won't upload: Error says "Could not find SKILL.md" — rename your file. Case matters.

Skill is slow: It might be too large. Move documentation to `references/` and link to it. Also, check how many skills you have enabled — more than 50 at once can slow things down.

MCP calls keep failing: Check that your MCP server is actually connected. Test it independently first: "Use [Service] to fetch my data." If that fails, the problem is MCP setup, not your skill.

Quick validation checklist

Before building: know your 2–3 main use cases, understand which tools you need, and review example skills similar to yours.

During building: folder named with dashes, SKILL.md spelled right, YAML has proper `---` delimiters, description covers what AND when, instructions are specific.

Before sharing: test on obvious triggers, test paraphrased requests, verify it doesn't fire on random topics, run the full workflow, check error handling.

After deployment, monitor usage, collect feedback, and iterate on the description and instructions.

What to do right now

Pick one workflow you keep explaining to Claude over and over. Use `skill-creator` to generate the first draft — takes about 20 minutes. Test it manually, fix what breaks, test again.

If it works, put it on GitHub.

Skills turn repetitive explanations into permanent knowledge. Build your first one this week. It's genuinely that useful.

Claude | Claude Code | Coding | Claude Skills | AI

Written by Civil Learning

6.1K followers · 6 following

We share what you need to know. Shared only for information.

Follow

Responses (5)

Kurt Anderson

What are your thoughts?

Kevin Lücke
2 days ago

The MCP vs Skills distinction is the clearest I've seen it explained — "what Claude CAN do" vs "how Claude SHOULD do it." That contractor/blueprint analogy nails why you need both layers. The three load levels (YAML frontmatter → SKILL.md body →... more

5 Reply

Ryck Birch
5 days ago

Tight summary. Nice work.

2 1 reply Reply

Andreas Thyholdt
1 day ago

Good info here, thanks. Btw, the minimal frontmatter block is malformed, it also contains commentary + the article text for chapter 3 and 4. Also, the later example showing missing dashes does in fact have a dash.

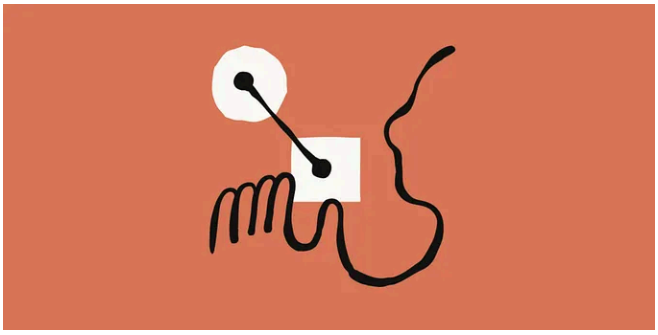
8 Reply

See all responses

More from Civil Learning

In Coding Nexus by Civil Learning

I Built an AI Company with OpenClaw + Vercel + Supabase —...

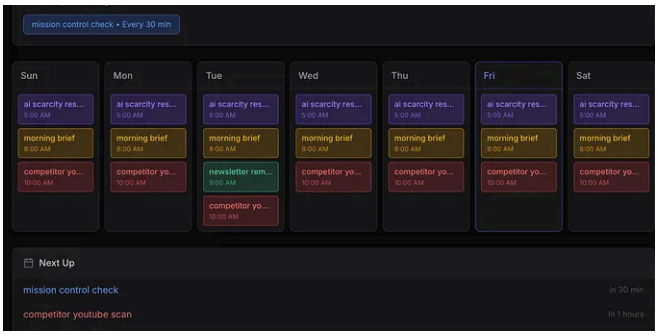


In Coding Nexus by Civil Learning

Claude Code Hooks: 5 Automations That Eliminate Developer Friction

6 AI agents, 1 VPS, 1 Supabase database — the journey from “agents can talk” to “agents...

Feb 11 513 18



In Coding Nexus by Civil Learning

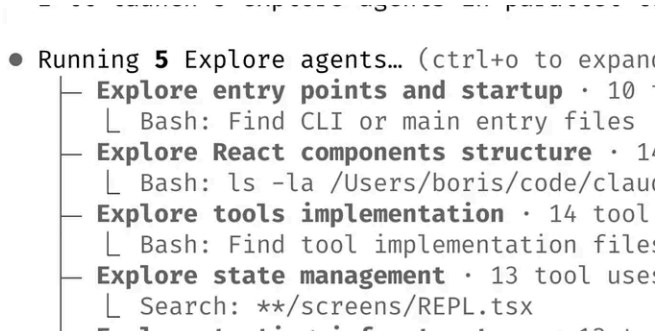
3 Things You Must Build Immediately With OpenClaw (If Yo...

If you’re running OpenClaw seriously, there’s a moment you eventually hit.

Feb 10 321 1

Claude Code hooks explained with real examples. Eliminate approvals, auto-format...

Jan 17 296 4



In Coding Nexus by Civil Learning

How the Claude Code Team Actually Codes

Inside the Claude Code workflow: parallel worktrees, plan mode, skills, subagents, and...

Feb 3 210 1

See all from Civil Learning

Recommended from Medium

In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Big Tech AI...

Something strange happened in early 2026. Apple stores started running low on Mac...

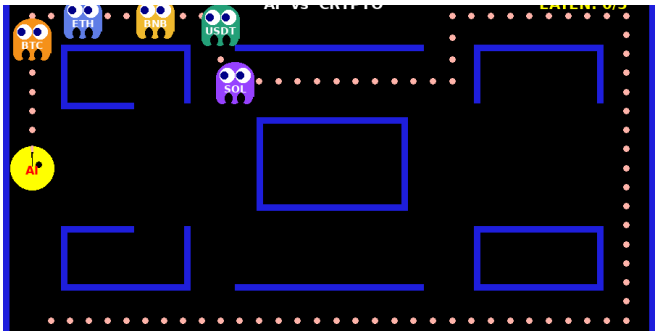
Feb 15 1.2K 21

In Activated Thinker by Shane Collins

Why the Smartest People in Tech Are Quietly Panicking Right Now

The water is rising fast, and your free version of ChatGPT is hiding the terrifying,...

Feb 13 9.8K 422



In AI Advances by Jose Crespo, PhD

Anthropic is Killing Bitcoin

The AI-native currency already exists — hiding in plain sight, outperforming crypto b...

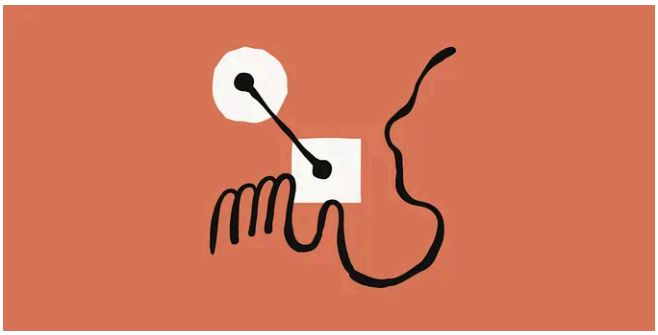
6d ago 1.5K 75

In AI Software Engineer by Joe Njenga

Claude Sonnet 4.6 Is Here : It Does Better Than Expensive Opus 4.6...

Sonnet 4.6 has been released barely days after Opus 4.6, and it feels like the better...

6d ago 249 4

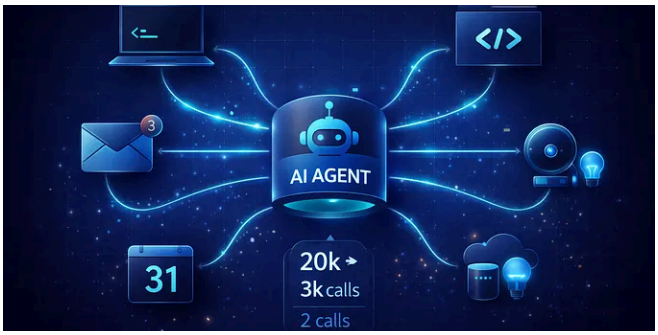


In Coding Nexus by Civil Learning

Claude Code Hooks: 5 Automations That Eliminate Developer Friction

Claude Code hooks explained with real examples. Eliminate approvals, auto-format...

Jan 17 296 4



Alex Rozdolskyi

10 Wild Things People Actually Built with OpenClaw

The wild, expensive, occasionally terrifying things people built with autonomous AI

Feb 12 45

See more recommendations